

OBJECT ORIENTED PROGRAMMING

Project Report

Canary Forensic Detection System

A Deceptive Cyber Defense Tool Built in C++

Submitted By:

Faaiza Mir (2503500)

Sana Rahat (2503539)

Dept: CyberSecurity

Program: BS-CYFS

Instructor: Miss Irum

Date: 30 / 05 / 2026

Contents

1. Introduction & Project Overview	3
1.1 Introduction.....	3
1.2 Project Overview	3
2. Problem Statement, Objectives & Scope.....	3
2.1 Problem Statement.....	3
2.2 Proposed Solution	3
2.3 Objectives.....	4
2.4 Scope	4
3. System Architecture & Design.....	4
3.1 Architectural Overview.....	4
3.2 System Flow & Mechanics.....	4
4. Class Hierarchy & OOP Design.....	6
4.1 Class Hierarchy	6
4.2 OOP Design Decisions	7
5. Key System Features.....	8
6. Challenges Faced & Lessons Learned.....	8
6.1 Challenges Faced.....	8
6.2 Lessons Learned	9
7. Future Improvements	9
8. Conclusion	10

1. Introduction & Project Overview

1.1 Introduction

In modern cybersecurity, reactive defense mechanisms often fall short when dealing with sophisticated, persistent network intruders. Traditional intrusion detection systems rely heavily on complex signature matching or heuristic anomalies, which can overlook zero day lateral movements. This project introduces the Canary Forensic Detection System: a deceptive defense mechanism designed around the classic intelligence concept of a honeytrap. By deploying digital bait, the system shifts the paradigm from passive monitoring to active, stealthy deception.

1.2 Project Overview

The Canary Forensic Detection System is a lightweight, low overhead security tool built in C++ using Object Oriented Programming principles. The software functions as a digital trap, strategically deploying decoy files (e.g., Passwords.txt) inside production environments to lure unauthorized users. Operating stealthily in the background, the application establishes a baseline structural snapshot of these traps. The moment an adversary interacts with, modifies, or deletes a decoy asset, the system instantly triggers forensic alerts and logs the event permanently for legal and incident response analysis.

2. Problem Statement, Objectives & Scope

2.1 Problem Statement

When an unauthorized actor breaches a network perimeter, tracking their lateral movement inside shared storage systems or local environments is exceptionally difficult without generating high volumes of false positive alerts. Security teams often lack immediate visibility into which files an insider or external hacker is targeting during the reconnaissance phase. Furthermore, standard logging systems can be altered or cleared by sophisticated attackers, making it hard to maintain an immutable, lightweight forensic trail of the initial compromise.

2.2 Proposed Solution

The proposed solution is an automated deception engine that establishes an unspoken contract within the filesystem. By populating a network shared folder with highly enticing, fake administrative assets, the system sets an invisible perimeter tripwire. It implements continuous background integrity validation using comparative cryptographic states. Because legitimate users have no operational reason to access these hidden bait configurations, any interaction guarantees a high fidelity indication of malicious intent, bypassing the noise of traditional log monitoring tools.

2.3 Objectives

- **Deceptive Asset Deployment**
Create distinct structural traps across individual files, folders, and network logs.
- **State Based Integrity Monitoring**
Design an object architecture capable of tracking file dimensions, timestamps, and data verification sums.
- **Asynchronous Detection**
Implement thread safe automated scanning to detect structural changes in real time without processing delays or system crashes.
- **Forensic Permanence**
Build an independent utility class to record immutable alert data directly to disk for post-incident audits.

2.4 Scope

The current implementation is focused on local filesystem monitoring and localized network shares (LAN environments). It provides real time detection of basic unauthorized file modifications, file creations, deletions, and mock server access attempts. The target deployment environment is geared toward individual endpoint monitoring or small scale laboratory workstations, establishing a foundational architecture that can be scaled to broader organizational network wrappers in the future.

3. System Architecture & Design

3.1 Architectural Overview

The system follows a highly modular, decoupled controller architecture split between two primary functional spaces: the Defender Space and the Hacker Space.

- **The Defender Space**
Responsible for establishing baseline snapshots, running the multi-threaded scanning engines, and evaluating operational state discrepancies.
- **The Hacker Space**
Comprises the network exposed interactive interfaces where an adversary triggers the traps via unauthorized file manipulation or remote server handshakes.

3.2 System Flow & Mechanics

The operational execution path follows a strict five-step loop:

- **Step 1: Deployment**
The system provisions decoy assets such as fake server files or credential lists into the designated directory path.
- **Step 2: Snapshotting**
The core engine reads the metadata and computes a unique verification checksum for each deployed asset, building a baseline data snapshot.

- **Step 3: Continuous Scanning**

A background thread periodically wakes up to re-evaluate the target properties without manual user intervention.

- **Step 4: Discrepancy Evaluation**

The tool compares current structural properties against the stored baseline state using an overloaded inequality check.

- **Step 5: Alert Isolation**

If an altered state is found, an immutable log entry containing timestamps and verification details is written to an external audit file.

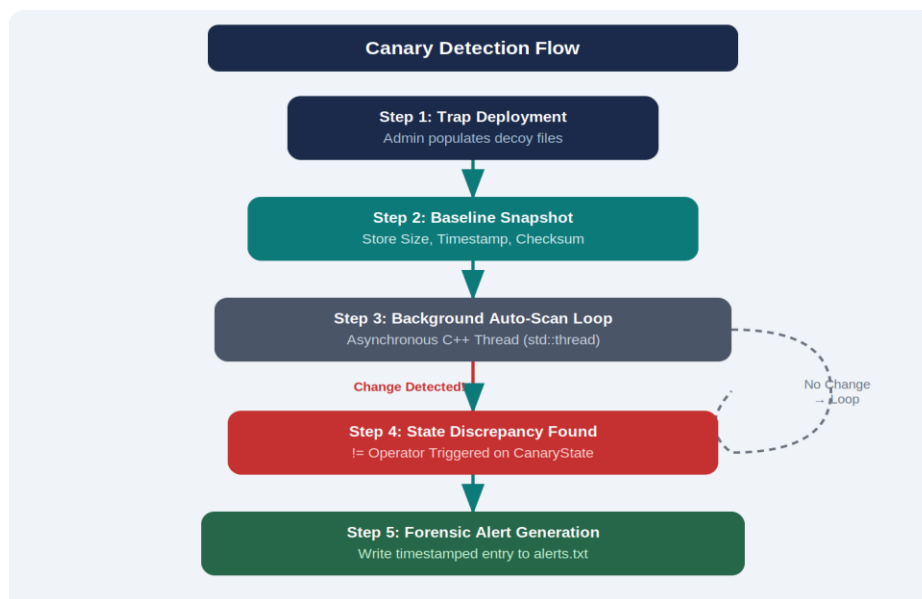


Figure 1 System Operational Architecture Flowchart

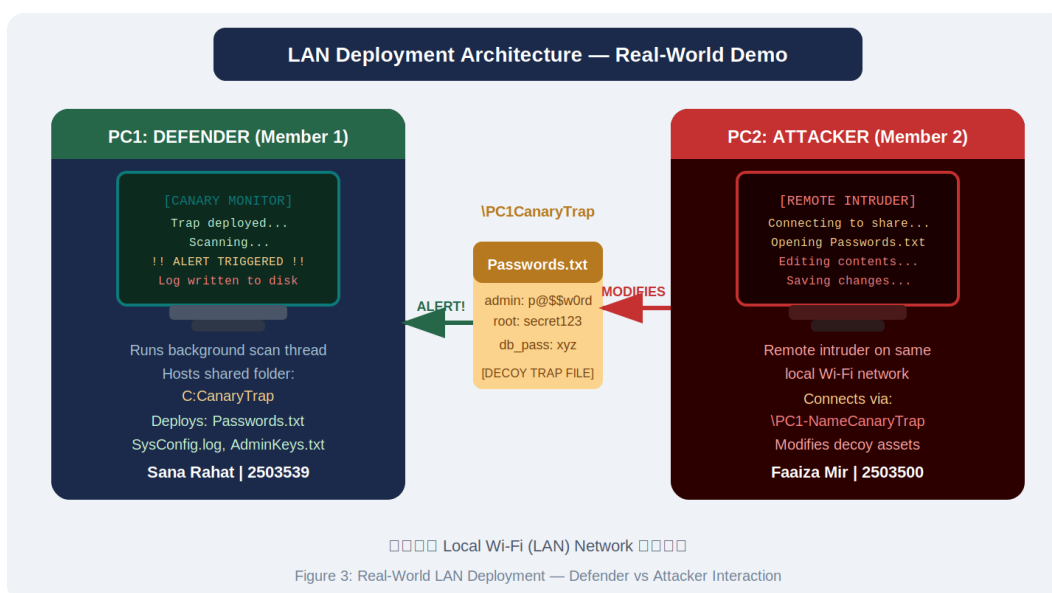


Figure 3: Real-World LAN Deployment — Defender vs Attacker Interaction

Figure 2 Real World LAN Deployment Architecture

4. Class Hierarchy & OOP Design

4.1 Class Hierarchy

The core architecture leverages a strongly typed hierarchy to allow seamless scaling for future decoy types. The following diagram illustrates the inheritance chain and class relationships:

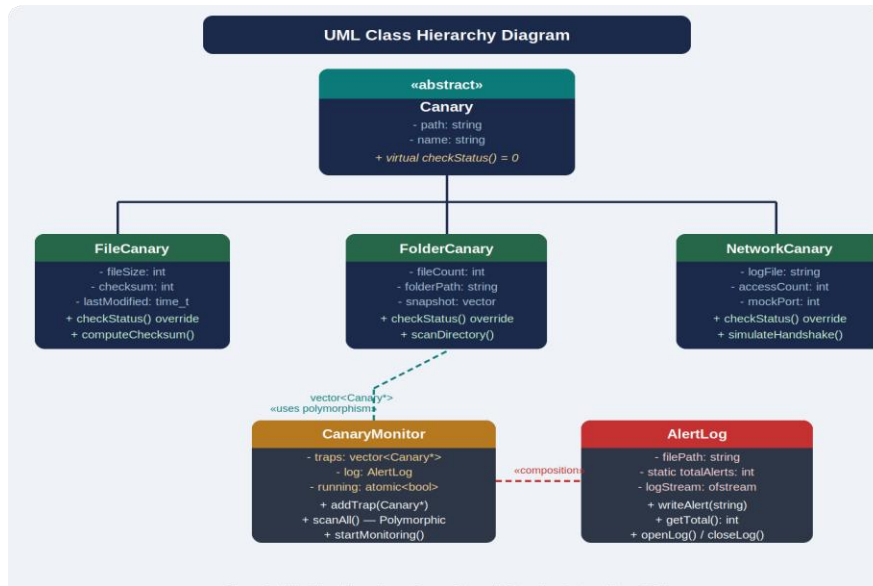


Figure 3 UML Class Hierarchy Diagram

Class Descriptions

- Canary (Abstract Base Class)**
 Serves as the blueprint for all trap types. Defines the mandatory interface rules and the pure virtual evaluation mechanics (`checkStatus() = 0`).
- FileCanary (Derived Class)**
 Implements targeted monitoring for single-file interactions by validating file size shifts and byte alterations via custom checksum computation.
- FolderCanary (Derived Class)**
 Oversees directories, tracking internal file counts and flagging unauthorized file additions or deletions within the monitored folder.
- NetworkCanary (Derived Class)**
 Emulates fake service communication entry points by tracking mock remote access logs and simulated TCP/IP handshake events.
- CanaryMonitor (Controller Class)**
 Contains a dynamic array of base Canary pointers to manage tracking configurations globally through polymorphic loops.
- AlertLog (Utility Class)**
 Handles direct system interaction for permanent, disk-based forensics. Maintains a static counter of total alerts triggered.

4.2 OOP Design Decisions

OOP Concept	Application in Source Code	Engineering Rationale
Classes & Objects	Applied across Canary, AlertLog, CanaryMonitor	Models every security component as an isolated operational object
Constructors / Destructors	Utilized in Canary and FileCanary subclasses	Automatically initialises target paths and captures baseline system states at creation
this Pointer	Used inside CanaryState code files	Avoids naming collisions by distinguishing class fields from function arguments
Static Members	Executed within AlertLog for totalAlerts counter	Shares an execution counter across all instances to track total compromises
Pointers to Objects	Employed in CanaryMonitor via vector<Canary*>	Dynamically stores and manages derived trap types at runtime
Operator Overloading	Overloaded != in the CanaryState structural block	Provides clean syntax to evaluate state snapshot alterations directly
Composition	CanaryMonitor HAS-A AlertLog (lifetime-bound)	Ties log management directly to the controller for automated error reporting
Inheritance	FileCanary, FolderCanary, NetworkCanary inherit from Canary	Reduces code duplication by reusing core identity traits and baseline parameters
Abstract Classes	Canary root class with pure virtual methods	Establishes a strict design contract; enforces consistency across all derived trap types
Polymorphism	Runtime dispatch in CanaryMonitor::scanAll() loop	Cycles through different trap variations uniformly via base pointers
File I/O	Leveraged inside AlertLog and file monitoring operations	Commits forensic evidence to disk (alerts.txt) so records persist after shutdown
Exception Handling	try-catch guards in main.cpp and file parsers	Keeps the monitoring loop active; prevents crashes if files are missing or deleted
Multi-Threading	Background worker thread in main.cpp	Runs automated scans asynchronously while keeping the console fully responsive

5. Key System Features

- **Polymorphic Multi-target Trapping**
Deploys file traps, directory traps, and network log simulators simultaneously under a single runtime environment.
- **Asynchronous Background Auto-Scanning**
Runs an independent execution thread that continuously monitors system integrity at scheduled intervals while keeping the primary console interactive.
- **State Integrity Verification Checksums**
Uses compound validations combining size constraints and character variations to detect data tampering even if file lengths match exactly.
- **Fault-Tolerant Exception Framework**
Isolates filesystem errors such as missing files or access denied warnings within structured try-catch guards to prevent application crashes.
- **Immutable Forensic Log Output**
Commits alert telemetry with high precision timestamps to an isolated storage file (alerts.txt) for persistent evidence preservation.

6. Challenges Faced & Lessons Learned

6.1 Challenges Faced

Deceptive Evaluation Flaws (Checksum Design)

Initially, the system only checked file sizes. Attackers could modify inner characters without shifting total byte counts, evading detection entirely. This was resolved by computing a cumulative character calculation to enforce strict inner-data integrity verification.

Abstract Implementation Obstacles

Defining pure virtual functions (`= 0`) created compile-time blocks when trying to directly instantiate generic parent objects. This highlighted the architectural rule that a base canary is simply a structural concept, and only concrete implementations should be instantiated and run.

PowerShell Wildcard Execution Failure

Compiling with standard multi-file commands (`src/*.cpp`) failed on Windows hosts because PowerShell does not natively resolve wildcard sequences the way Linux environments do. The issue was resolved by explicitly defining all relative compiler paths within the build script.

Thread Synchronization and Race Conditions

Running asynchronous scanning routines while keeping the user interface active caused resource collision risks. This was addressed by introducing thread-safe atomic flags (`std::atomic<bool>`) to manage clean background closures without data races.

6.2 Lessons Learned

- **Practical Value of Object Architectures:** OOP is more than abstract theory; polymorphic vectors let developers manage diverse security assets through simplified, scalable loops.
- **Architectural Contracts:** Abstract interfaces guarantee that any future honeytrap module will automatically align with the tracking engine's design contract.
- **Forensic File Strategy:** Hands-on file I/O operations exposed the critical real-world demands of writing unalterable, structured historical event timelines.
- **Industry Concept Realization:** The internal logic aligns closely with corporate deception networks like Canarytokens.org, proving how simple, elegant software patterns power real-world defensive engineering.

7. Future Improvements

To scale this implementation into an enterprise-grade utility, the following technical enhancements are planned:

Target Module	Technical Engineering Plan	Expected Defensive Value
SHA-256 Checksum Engine	Integrate true cryptographic hashing instead of character summations	Eliminates hash collision risks; validates true byte integrity
Automated Email Alerts	Connect SMTP APIs to trigger outward communication on mutation flags	Dispatches critical incident alerts to off-site SOC response teams
Attacker IP Logger	Build network socket sniffing modules to catch TCP/IP streams	Logs hardware identifier, network interface, and IP of the attacker
Encrypted Forensic Logs	Implement AES-256 block encryption for alerts.txt records	Prevents intruders from deleting or modifying their tracking footprints
Database Storage Migration	Transition from plain text to local SQLite database architecture	Enables secure data indexing, faster log processing, and metadata search
Administrative Graphical UI	Replace terminal CLI with a Qt-based user application wrapper	Provides investigators with an intuitive visual control panel for honeytraps

8. Conclusion

The Canary Forensic Detection System successfully demonstrates the practical power of Object-Oriented Programming in solving real-world cybersecurity challenges. By translating theoretical OOP concepts — inheritance, polymorphism, abstract classes, composition, and multi-threading — into a functioning deception-based security tool, this project bridges the gap between academic learning and industry-grade defensive engineering.

The system's modular architecture ensures that each security component operates independently while contributing to a unified detection pipeline. The polymorphic trap management model allows the system to monitor files, folders, and network activities through a single interface, demonstrating the scalability advantages of well-designed class hierarchies.

The challenges encountered during development — from checksum design flaws to thread synchronization — provided invaluable hands-on experience with the complexities of real-time systems programming. Each obstacle led to more robust, fault-tolerant design decisions that strengthened the overall architecture.

This foundation can be directly scaled toward enterprise-grade honeypot frameworks with the planned additions of SHA-256 cryptographic hashing, encrypted forensic logs, automated SOC alerting, and a graphical administrative interface — establishing the Canary Forensic Detection System as a meaningful contribution to the field of proactive cyber defense.